

Setup Goldengate for Postgres (Oracle Hash to Aurora Postgres)

Pre-requisites

- Installing and setting up Oracle GoldenGate connecting to an Oracle database
- Installing and setting up Oracle GoldenGate on the Postgres machine

1.1. 1) Database configurations pre-requisite on Postgres

The database configurations pre-requisite are very minimal. The following attributes in the PostgreSQL configuration file, located at \$PG_INSTALL_HOME/data/postgresql.conf file, need to be modified as follows:

```
wal_level = logical          # minimal, replica, or logical
max_replication_slots = 10   # max number of replication slots
```

Note: After any kind of changes made to the postgresql.conf configuration file, the database needs to be restarted.

1.2. 2) Configuring Extract from PostgreSQL and Replicating to PostgreSQL or Oracle Target Database

1.2.1. A) Create a database user, having replication user privileges, that is dedicated to Oracle GoldenGate.

```
create user ggs with password 'XXXXXXXXX';
create user ggt with password 'XXXXXXXXX';

GRANT rds_replication TO ggs;
GRANT rds_replication TO ggt;

create schema ggschema;

grant usage, create on schema ggschema ,pspadm to ggt;
grant usage, create on schema ggschema ,pspadm to ggs;
```

1.2.2. B) Enabling Supplemental Logging for a Source PostgreSQL Database

There are four levels of table level logging in PostgreSQL, which equate to the REPLICA IDENTITY setting of a table, and those include NOTHING, USING INDEX, DEFAULT, and FULL.

Oracle GoldenGate requires FULL logging for use cases that require uncompressed trail records and Conflict Detection and Resolution, but in cases where tables have a Primary Key or Unique Index whose changes are being replicated in a simple uni-directional configuration or where full before-images or uncompressed records are not needed, then the DEFAULT level is acceptable. NOTHING and USING INDEX logging levels are not supported by Oracle GoldenGate and cannot be set with ADD TRANDATA

In our Case, We have used the replica Identity FULL, Use below Command to enable the Replica Identity.

```
ALTER TABLE owner.mytable REPLICA IDENTITY FULL;
```

```
select 'alter table '|| schemaname || '.'|| tablename ||' replica identity full;' from pg_tables where  
schemaname='pspadm'
```

Changing the replica identity will not cause down time, but it requires a short ACCESS EXCLUSIVE lock on the table, which can be a problem if you have long running transactions that involve the table.

Other ways to enable supplemental logging from golden gate is as below.

```
GGSCI > DBLOGIN sourcedb pspapg02 USERID ggs PASSWORD "XXXXXXXXX"  
GGSCI > ADD TRANDATA dbo.table1  
GGSCI > ADD TRANDATA dbo.table2  
GGSCI > ADD TRANDATA public.table1 [ALLCOLS][KEYCOLSONLY]
```

Example :-

```
GGSCI > DBLOGIN SOURCEDB pspapg01, USERID ggt, PASSWORD "XXXXXXXXXX"  
GGSCI > ADD TRANDATA pspadm.arch_purge_main_stats_list KEYCOLSONLY  
GGSCI > ADD TRANDATA pspadm.as400_dropme keycolsonly
```

Dynamic Query to generate the Trandata script for all the tables schematise.

```
SQL >> SELECT 'ADD TRANDATA pspadm.'|| tablename || ' keycolsonly' FROM pg_tables where schemaname in  
('pspadm');
```

1.2.3. C) Create Read- Write Roles and Assing to the GGT and GGs users as below.

```
create role pspadm_readwrite_role;  
grant connect on database pspapg02 to pspadm_readwrite_role;  
grant usage on schema pspadm to pspadm_readwrite_role;  
grant select, insert, update, delete on all tables in schema pspadm to pspadm_readwrite_role;  
alter default privileges in schema pspadm grant select, insert, update, delete on tables to  
pspadm_readwrite_role;  
grant usage on all sequences in schema pspadm to pspadm_readwrite_role;  
alter default privileges in schema pspadm grant usage on sequences to pspadm_readwrite_role;  
  
Grant The Role to GGS and GGT users  
  
grant pspadm_readwrite_role to ggs,ggt;
```

1.2.3 D) Disable The Constraints while replicating since we are using Co-ordinated Replicat. Disable only for GGT users using below command.

```
alter user ggt set session_replication_role='replica';
```

1.2.4. D) Create Extract and Register to the Database

Before performing the Extract setup, consider that the PostgreSQL's LIBPQ(\$PG_INSTALL_HOME/lib) library and the DataDirect ODBC driver (\$OGG_HOME/lib) are required to capture data from PostgreSQL database. The DataDirect ODBC driver is shipped with the OGG shiphome. Hence, no need to separately download the ODBC driver.

Here are the steps to configure the Extract:

- **Create the ODBC.ini file. See the following sample \$ODBCINI file**

Create the ODBC file in GG_HOME

```
$$ export ODBCINI=/u01/ogg/pspag01/postgres/21.9.0.0.3/odbc.ini
```

Put the entry into //oggpg file

```
$$ . //oggpg
```

- **Create Extract. ([EPSPPG02.prm](#))**

Create Extrcat Parameter file and register the extract

```
GGSCI > DBLOGIN SOURCEDB pspapg02, USERID ggs, PASSWORD "XXXXXXX"
GGSCI > register extract EPSPPG02 with database pspapg02
```

Add the Extract to the GG

```
GGSCI > ADD EXTRACT EPSPPG02, TRANLOG, BEGIN NOW, desc 'Postgres Via Range Replicat'
GGSCI> ADD EXTTRAIL ./dirdat/pspag02/tr, EXTRACT EPSPPG02, megabytes 1000
```

Start Extract

```
GGSCI> DBLOGIN SOURCEDB pspapg02, USERID ggt, PASSWORD "XXXXXXXXXX"
GGSCI> START EXTRACT EPSPPG02
```

1.3. e) Create Replicat and Register to the Database

- **Database Pre-requisites**

Create a database user, having replication user privileges, that is dedicated to Oracle GoldenGate.

```
create user ggs with password 'xxx';
create user ggt with password 'xxx';
```

Grant replication privileges to the users.

```
GRANT rds_replication TO ggs;
GRANT rds_replication TO ggt;
```

- **Create the odbc.ini file**

Golden Gate uses an ODBC connection to connect to the Postgres database. The ODBC driver is shipped with the installation and on Unix you have to create the ODBC configuration file which is commonly called odbc.ini on your own. You need to create this file in GG_HOME directory on the target system.

Once the ODBC file is created, we need to export an ODBC environment variable which is called ODBCINI and points to the odbc.ini file

```
$$ export ODBCINI=$GG_HOME/odbc.ini
```

NOTE: POSTGRES SPECIAL

Security at Postgres may deny connections from other hosts, so check the Postgres config files:

Postgres conf file pg_hba.conf needs this config line so that ALL clients can connect.

host	all	all	0.0.0.0/0	md5
------	-----	-----	-----------	-----

A second config file is the Listener which is configured in the postgresql.conf. The parameter:

```
listen_addresses '*'
listen_addresses = 'localhost'          # what IP address(es) to listen on;
```

so only localhost connections are possible.

- **Set the environmental variables to point locations for odbc file along with lib directory and installation dir and create all required GG specific directories on the target database server.**

```
$$ export ODBCINI=/home/postgres/ggdirpsql/odbc.ini
$$ export LD_LIBRARY_PATH=/home/postgres/ggdirpsql/lib
$$ ./ggsci
```

```
GGSCI > create subdirs
GGSCI > edit param mgr
GGSCI > start mgr
GGSCI > dblogin sourcedb PSPAPG01 userid ggt
GGSCI > list tables pspadm.*
GGSCI > view params RPSPPG01
```

So we can successfully connect to the Oracle database and to the Postgres database. Both connections are mandatory. Do not continue with the next steps unless both connections are working.

- **Create Replicat. ([RPSPP03.prm](#))**

```
GGSCI > DBLOGIN SOURCEDB PSPAPG01, USERID ggt, PASSWORD "XXXXX"
```

```
GGSCI > ADD CHECKPOINTTABLE ggschema.checkpoint
```

```
GGSCI > add replicat RPSPHP02, exttrail /u01/ogg/psphpp05/oracle/21.9.0.0.0/dirdat/psphnp02/tr,  
CHECKPOINTTABLE ggschema.checkpoint
```

Please feel free to add if something is missed or seems relevant !!!